Fondements de la programmation - Partiel 1

Documents autorisés - durée 3h - barême donné à titre indicatif

On considère le mini-langage suivant.

TYPE

τ ::= num (type des entiers)
opt τ (type option)

EXPRESSION

e ::= x (variable)
n (entier littéral)
$e_1 + e_2$ (addition)
e_1 / e_2 (division)
`let` x=e_1 `in` e_2 (définition)
`None` (absence)
`Some` e (présence)
`match` e_1 `with None=>` e_2 | `Some` x `=>` e_3 (analyse de cas)

Priorités :

(définition) < (analyse de cas) < (présence) < (addition) < (division)

Associativité à gauche :
addition, division

Lieurs :
définition : x dans e_2
analyse de cas : x dans e_3

Les constructeurs `None` et `Some` x permettent de signaler l'absence ou la présence d'un résultat. Par exemple, le resultat de 4/0 est `None` (pas de résultat), et le résultat de 4/2 est `Some` 2 (il y a résultat, et ce résultat est 2). Le constructeur `match` e_1 `with None=>` e_2 | `Some` x `=>` e_3 permet de faire une analyse de cas sur ces deux possibilités.

Syntaxe

Question 1 (1 pt) – Donner la définition de l'ensemble des variables libres d'une expression fv(e) pour ce langage.

Question 2 (3 pts) – Écrire les arbres lieurs abstraits (ABT) des expressions suivantes :
— `let` x=12 `in Some` x+3/5
— `let` x=`Some` 12 `in match` x `with None=>` x | `Some` x `=>` x+1

Types

Voici le système de types de ce langage.

STR
$$\frac{}{\Gamma \vdash n : \text{num}}$$

VAR
$$\frac{}{\Gamma, x:\tau \vdash x : \tau}$$

NONE
$$\frac{}{\Gamma \vdash \texttt{None} : \text{opt } \tau}$$

ADD
$$\frac{\Gamma \vdash e_1 : \text{num} \qquad \Gamma \vdash e_2 : \text{num}}{\Gamma \vdash e_1 + e_2 : \text{num}}$$

DIV
$$\frac{\Gamma \vdash e_1 : \text{num} \qquad \Gamma \vdash e_2 : \text{num}}{\Gamma \vdash e_1 / e_2 : \text{opt num}}$$

SOME
$$\frac{\Gamma \vdash e : \tau}{\Gamma \vdash \texttt{Some } e : \text{opt } \tau}$$

LET
$$\frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x:\tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \texttt{let } x=e_1 \texttt{ in } e_2 : \tau_2}$$

MATCH
$$\frac{\Gamma \vdash e_1 : \text{opt } \tau_1 \qquad \Gamma \vdash e_2 : \tau_2 \qquad \Gamma, x:\tau_1 \vdash e_3 : \tau_2}{\Gamma \vdash \texttt{match } e_1 \texttt{ with None=> } e_2 \texttt{ | Some } x \texttt{ => } e_3 : \tau_2}$$

Scanned with CamScanner

Question 3 (4 pts) – Les expressions suivantes sont-elles typables ? Si oui, écrire la dérivation de typage ; sinon, expliquer rapidement pourquoi.

- $(12/5)+8$
- `Some None`
- `match` $12/3$ `with None=> None | Some` x `=>` x
- `let` x`=Some 12 in (match` x `with None=>` x `| Some` y `=>` $y/0$)

Question 4 (1 pt) – Montrer que si une expression est typable, son type n'est pas forcément unique.

Dynamique

On donne la définition de substitution et la sémantique opérationnelle structurelle.

$$x\{f/x\} = f \qquad (e_1 + e_2)\{f/x\} = e_1\{f/x\} + e_1\{f/x\}$$
$$y\{f/x\} = y \text{ si } x \neq y \qquad (e_1 / e_2)\{f/x\} = e_1\{f/x\} / e_1\{f/x\}$$
$$\texttt{None}\{f/x\} = \texttt{None} \qquad (\texttt{let } y{=}e_1 \texttt{ in } e_2)\{f/x\} = \texttt{let } y{=}e_1\{f/x\} \texttt{ in } e_2\{f/x\} \text{ si } y \neq x \text{ et } y \notin \text{fv}(f)$$
$$(\texttt{Some } e)\{f/x\} = \texttt{Some } e\{f/x\}$$
$$(\texttt{match } e_1 \texttt{ with None=> } e_2 \texttt{ | Some } y \texttt{ => } e_3)\{f/x\} = \texttt{match } e_1\{f/x\} \texttt{ with None=> } e_2\{f/x\} \texttt{ | Some } y \texttt{ => } e_3\{f/x\}$$
$$\text{si } y \neq x \text{ et } y \notin \text{fv}(f)$$

ADDG
$$\frac{e_1 \to e_1'}{e_1 + e_2 \to e_1' + e_2}$$

ADDD
$$\frac{e_2 \to e_2'}{v + e_2 \to v + e_2}$$

ADD
$$\frac{n_1 + n_2 = n}{n_1 + n_2 \to n}$$

DIVG
$$\frac{e_1 \to e_1'}{e_1 / e_2 \to e_1' / e_2}$$

DIVD
$$\frac{e_2 \to e_2'}{v / e_2 \to v / e_2}$$

DIV
$$\frac{n_1 / n_2 = n}{n_1 / n_2 \to \texttt{Some } n}$$

DIV0
$$\frac{}{n_1 / 0 \to \texttt{None}}$$

SOME
$$\frac{e \to e'}{\texttt{Some } e \to \texttt{Some } e'}$$

LET
$$\frac{e_1 \to e_1'}{\texttt{let } x{=}e_1 \texttt{ in } e_2 \to \texttt{let } x{=}e_1' \texttt{ in } e_2}$$

LETV
$$\frac{}{\texttt{let } x{=}v \texttt{ in } e_2 \to e_2\{v/x\}}$$

MATCH
$$\frac{e_1 \to e_1'}{\texttt{match } e_1 \texttt{ with None=> } e_2 \texttt{ | Some } x \texttt{ => } e_3 \to \texttt{match } e_1' \texttt{ with None=> } e_2 \texttt{ | Some } x \texttt{ => } e_3}$$

MATCHNONE
$$\frac{}{\texttt{match None with None=> } e_2 \texttt{ | Some } x \texttt{ => } e_3 \to e_2}$$

MATCHSOME
$$\frac{}{\texttt{match Some } v \texttt{ with None=> } e_2 \texttt{ | Some } x \texttt{ => } e_3 \to e_3\{v/x\}}$$

Les valeurs sont $v ::= n \mid x \mid \texttt{None} \mid \texttt{Some } v$.

Question 5 (2 pts) – Réduire les expressions suivantes jusqu'à obtenir une valeur, en donnant les étapes intermédiaires (sans donner les arbres de réduction).

- `let` x`=12/0 in (match` x `with None=> 0 | Some` y `=>` $y+1$)
- `(match Some 12 with None=> 3 | Some` x `=>` $2+x$`)/0`

Question 6 – (4 pts) Montrer que pour tout e, f, v, y et x, si $x \neq y$ et $x \notin \text{fv}(v)$, alors

$$e\{f/x\}\{v/y\} = e\{v/y\}\{f\{v/y\}/x\}.$$

Lister tous les cas de la preuve par induction, mais ne développer que les cas de bases intéressants, un cas par induction sans lieur, et un cas par induction avec lieur.

Scanned with CamScanner

Définition avec option

Pour enchaîner plus facilement les calculs qui peuvent échouer, on introduit la définition avec option `let` x`=?`e_1 `in` e_2, qui peut être définie à l'aide des constructeurs déjà présents dans le langage :

`let` x`=?`e_1 `in` e_2 est défini comme `match` e_1 `with None=> None | Some` x `=>` e_2

Question 7 – (1 pt) Donner la valeur finale de l'évaluation des expressions suivantes.

— `let` x`=?4/2 in let` y`=?6/3 in` x`/`y → Some 1 = 1

— `let` x`=?4/2 in let` y`=?6/0 in` x`/`y

Question 8 – (4 pts)

— Écrire la ou les règle(s) de typage de la définition avec option.

— Écrire la ou les règle(s) de réduction de la définition avec option.

Scanned with CamScanner